

Task 1: Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.

```
shreyas@Grigori:~$ mkdir -p ~/memory_allocation
shreyas@Grigori:~$ cd ~/memory_allocation
shreyas@Grigori:~/memory_allocation$ nano memory.c
shreyas@Grigori:~/memory_allocation$ gcc -g memory.c -o memory
shreyas@Grigori:~/memory_allocation$ ./memory
Memory allocated using malloc():
1 2 3 4 5 6 7 8 9 10

Memory allocated using calloc():
10 20 30 40 50

Memory allocation using realloc() completed.
shreyas@Grigori:~/memory_allocation$ valgrind --leak-check=full ./memory
==1018== Memcheck, a memory error detector
==1018== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==1018== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==1018== Command: ./memory
==1018==
Memory allocated using malloc():
1 2 3 4 5 6 7 8 9 10

Memory allocated using calloc():
10 20 30 40 50

Memory allocation using realloc() completed.
==1018==
==1018== HEAP SUMMARY:
==1018==    in use at exit: 400 bytes in 1 blocks
==1018== total heap usage: 5 allocs, 4 frees, 1,504 bytes allocated
==1018==
==1018== 400 bytes in 1 blocks are definitely lost in loss record 1 of 1
==1018==    at 0x4846828: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==1018==    by 0x109315: main (memory.c:41)
==1018==
==1018== LEAK SUMMARY:
==1018==    definitely lost: 400 bytes in 1 blocks
==1018==    indirectly lost: 0 bytes in 0 blocks
==1018==    possibly lost: 0 bytes in 0 blocks
==1018==    still reachable: 0 bytes in 0 blocks
==1018==    suppressed: 0 bytes in 0 blocks
==1018==
==1018== For lists of detected and suppressed errors, rerun with: -s
```

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *a;
    int *b;
    int *c;

    a = malloc(5 * sizeof(int));

    if (a == NULL) {
        return 1;
    }

    for (int i = 0; i < 5; i++) {
        a[i] = i + 1;
    }

    b = calloc(5, sizeof(int));

    if (b == NULL) {
        free(a);
        return 1;
    }

    for (int i = 0; i < 5; i++) {
        b[i] = (i + 1) * 10;
    }
}
```

```

a = realloc(a, 10 * sizeof(int));

if (a == NULL) {
    free(b);
    return 1;
}

for (int i = 5; i < 10; i++) {
    a[i] = i + 1;
}

c = malloc(100 * sizeof(int));

if (c == NULL) {
    free(a);
    free(b);
    return 1;
}

printf("Memory allocated using malloc():\n");
for (int i = 0; i < 10; i++) {
    printf("%d ", a[i]);
}

printf("\n\nMemory allocated using calloc():\n");
for (int i = 0; i < 5; i++) {
    printf("%d ", b[i]);
}

```

```

printf("Memory allocated using malloc():\n");
for (int i = 0; i < 10; i++) {
    printf("%d ", a[i]);
}

printf("\n\nMemory allocated using calloc():\n");
for (int i = 0; i < 5; i++) {
    printf("%d ", b[i]);
}

printf("\n\nMemory allocation using realloc() completed.\n");

free(a);
free(b);

return 0;
}

```

```
shreyas@Grigori:~/memory_allocation$ |
```

Task 2: Develop a program that demonstrates Copy-on-Write behavior after a fork(). Analyze memory usage before and after modifying data in the child process.

```
shreyas@Grigori:~/memory_allocation$ mkdir -p ~/copy_on_write
shreyas@Grigori:~/memory_allocation$ cd ~/copy_on_write
shreyas@Grigori:~/copy_on_write$ nano cow.c
shreyas@Grigori:~/copy_on_write$ gcc -O0 -g cow.c -o cow
shreyas@Grigori:~/copy_on_write$ ./cow
```

Parent before fork:

Parent - PID: 1049

Rss:	5968 kB
Pss:	4295 kB
Pss_Dirty:	4212 kB
Pss_Anon:	4200 kB
Pss_File:	95 kB
Pss_Shmem:	0 kB
Shared_Clean:	1756 kB
Shared_Dirty:	0 kB
Private_Clean:	0 kB
Private_Dirty:	4212 kB
Shared_Hugetlb:	0 kB
Private_Hugetlb:	0 kB

Child after fork, before modification:

Child - PID: 1052

Rss:	5232 kB
Pss:	2168 kB
Pss_Dirty:	2122 kB
Pss_Anon:	2118 kB
Pss_File:	50 kB

Child after fork, before modification:

Child - PID: 1052

Rss:	5232 kB
Pss:	2168 kB
Pss_Dirty:	2122 kB
Pss_Anon:	2118 kB
Pss_File:	50 kB
Pss_Shmem:	0 kB
Shared_Clean:	1024 kB
Shared_Dirty:	4172 kB
Private_Clean:	0 kB
Private_Dirty:	36 kB
Shared_Hugetlb:	0 kB
Private_Hugetlb:	0 kB

Child modifying data...

Child after modification:

Child - PID: 1052

Rss:	5232 kB
Pss:	4218 kB
Pss_Dirty:	4172 kB
Pss_Anon:	4168 kB
Pss_File:	50 kB
Pss_Shmem:	0 kB
Shared_Clean:	1024 kB
Shared_Dirty:	72 kB

```
Child - PID: 1052
Rss:          5232 kB
Pss:          4218 kB
Pss_Dirty:    4172 kB
Pss_Anon:     4168 kB
Pss_File:     50 kB
Pss_Shmem:    0 kB
Shared_Clean: 1024 kB
Shared_Dirty: 72 kB
Private_Clean: 0 kB
Private_Dirty: 4136 kB
Shared_Hugetlb: 0 kB
Private_Hugetlb: 0 kB
```

Parent after child finishes:

```
Parent - PID: 1049
Rss:          5968 kB
Pss:          4295 kB
Pss_Dirty:    4212 kB
Pss_Anon:     4200 kB
Pss_File:     95 kB
Pss_Shmem:    0 kB
Shared_Clean: 1756 kB
Shared_Dirty: 0 kB
Private_Clean: 0 kB
Private_Dirty: 4212 kB
Shared_Hugetlb: 0 kB
Private_Hugetlb: 0 kB
```

```
shreyas@Grigori:~/copy_on_write$ |
```

```
shreyas@Grigori:~/copy_on_write$ cat cow.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

#define SIZE (1024 * 1024)

void show_memory(const char *name) {
    char command[256];

    snprintf(command, sizeof(command),
             "grep -E '^(Rss|Pss|Private|Shared)' /proc/%d/smaps_rollup",
             getpid());

    printf("\n%s - PID: %d\n", name, getpid());
    fflush(stdout);

    system(command);
}

int main() {
    int *data = malloc(SIZE * sizeof(int));

    if (data == NULL) {
        perror("malloc");
        return 1;
    }

    for (int i = 0; i < SIZE; i++) {
```

```

    if (pid == 0) {
        printf("\nChild after fork, before modification:\n");
        show_memory("Child");

        printf("\nChild modifying data...\n");

        for (int i = 0; i < SIZE; i++) {
            data[i] = 2;
        }

        printf("\nChild after modification:\n");
        show_memory("Child");

        free(data);
        exit(0);
    } else {
        wait(NULL);

        printf("\nParent after child finishes:\n");
        show_memory("Parent");

        free(data);
    }

    return 0;
}

```